Portada

Entregable 2

Nombre estudiante

Gustavo Mata Avellan

Kevin Ramírez Soto

Universidad Politécnica

Curso: Tecnicas de Programacion

Profesor: Luis Felipe Mora Umaña

28 de abril del 2026

Índice

Introducción.....3
Desarrollo.....4
EPIC 1. Migración a Blazor Server + SQLite.....4
1.1 Feature: Migración de la capa de presentación.....5
1.1.1 PBI: Cambio de capa View (WinForms → Blazor).....5
1.1.1.1 Task: Cambio de capa View.....5
1.1.2 PBI: Migración de vistas a Razor.....5
1.1.2.1 Task: Migración de vistas.....5
1.2 Feature: Persistencia de datos con SQLite.....5
1.2.1 PBI: Implementación de base de datos.....5
1.2.1.1 Task: Configuración de SQLite.....6
1.2.2 PBI: Migración de controllers a SQLite.....6
1.2.2.1 Task: Migración de lógica de datos.....6
1.3 Feature: Ajustes del modelo y estado de la aplicación.....6
1.3.1 PBI: Actualización del modelo User.....6
1.3.1.1 Task: Extensión del modelo.....6
1.3.2 PBI: Manejo de estado de sesión.....7
1.3.2.1 Task: Implementación de estado de sesión.....7
EPIC 2. Separación de Controllers.....7
2.1 Feature: Refactorización basada en principios SOLID.....7
2.1.1 PBI: Separación de responsabilidades en controllers.....7
2.1.1.1 Task: REFACTOR-1.1 — Separar LoginController en dos clases.....7
EPIC 3. Implementar panel de administración.....8
3.1 Feature: Interfaz de administración.....8
3.1.1 PBI: Vista de administración.....8
3.1.1.1 Task: ADMIN-1.1 — Vista de administración.....8
3.2 Feature: Estadísticas administrativas.....8
3.2.1 PBI: Visualización de estadísticas globales.....8
3.2.1.1 Task: ADMIN-1.2 — Estadísticas administrativas.....8
3.3 Feature: Integración en navegación.....9
3.3.1 PBI: Acceso al panel de administración.....9
3.3.1.1 Task: ADMIN-1.3 — Integración en navegación.....9
EPIC 4. Mejora — Testing y Cobertura de Controllers.....9
4.1 Feature: Configuración del entorno de pruebas.....9
4.1.1 PBI: Implementación del proyecto de testing.....10
4.1.1.1 Task: TEST-1 — Configurar proyecto de pruebas xUnit.....10
4.2 Feature: Pruebas unitarias de controllers.....10
4.2.1 PBI: Validación funcional del sistema.....10
4.2.1.1 Task: TEST-2 — Tests de LoginController.....10
4.2.1.2 Task: TEST-3 — Tests de AlimentoController.....11
4.2.1.3 Task: TEST-4 — Tests de PerfilController.....11
4.2.1.4 Task: TEST-5 — Tests de MenuController.....12

4.2.1.5 Task: TEST-6 — Tests de UserManagementController..... 12
5. Descarga de Reportes..... 13
5.1 Feature: Exportación desde Menús..... 13
5.1.1 PBI: Descarga CSV..... 13
5.1.1.1 Task: CSV-1..... 13
5.2 Feature: Exportación desde Estadísticas..... 13
5.2.1 PBI: Descarga CSV en estadísticas..... 13
5.2.1.1 Task: CSV-2..... 13
Conclusiones / Análisis de Aprendizaje..... 14

Introducción

En el desarrollo de aplicaciones modernas, la calidad del software, la mantenibilidad y la escalabilidad son aspectos fundamentales para garantizar un sistema robusto y confiable. En este contexto, el presente proyecto aborda la evolución de una aplicación de gestión nutricional mediante la adopción de buenas prácticas de ingeniería de software, incluyendo la refactorización de la arquitectura, la implementación de patrones de diseño y la incorporación de pruebas automatizadas.

Inicialmente, el sistema presentaba un acoplamiento directo entre la lógica de negocio y el acceso a datos, así como una ausencia de mecanismos formales de validación mediante pruebas. Para solventar estas limitaciones, se implementó el patrón Repository, permitiendo separar las responsabilidades y mejorar la organización del código. Asimismo, se integró una estrategia de testing utilizando xUnit y una base de datos SQLite en memoria.

Adicionalmente, se incorporaron mejoras funcionales orientadas al usuario final, como la generación y descarga de reportes en formato CSV.

Desarrollo

EPIC 1. Migración a Blazor Server + SQLite

Descripción:
Este Epic tiene como objetivo modernizar la arquitectura del sistema mediante la migración de una aplicación de escritorio basada en WinForms hacia una aplicación web utilizando Blazor Server. Además, se sustituye el uso de archivos CSV por una base de datos SQLite, mejorando la persistencia, escalabilidad y organización de los datos.

1.1 Feature: Migración de la capa de presentación

1.1.1 PBI: Cambio de capa View (WinForms → Blazor)

1.1.1.1 Task: Cambio de capa View

Descripción:
Eliminar completamente la interfaz basada en WinForms y configurar la nueva estructura de Blazor Server como base de la aplicación web.

- 1.1.1.1.1 Subtask: Eliminar archivos Frm*.cs de la carpeta Views
- 1.1.1.1.2 Subtask: Crear proyecto Blazor Server (.NET 8)
- 1.1.1.1.3 Subtask: Modificar NutricionApp.csproj de WinExe a Web
- 1.1.1.1.4 Subtask: Crear Program.cs con configuración inicial de Blazor
- 1.1.1.1.5 Subtask: Crear App.razor, _Imports.razor, Shared/MainLayout.razor y Pages/_Host.cshtml

---

1.1.2 PBI: Migración de vistas a Razor

1.1.2.1 Task: Migración de vistas

Descripción:
Reemplazar todas las interfaces de usuario desarrolladas en WinForms por componentes Razor en Blazor Server, manteniendo la funcionalidad original del sistema.

- 1.1.2.1.1 Subtask: Crear Views/Login.razor (reemplaza FrmLogin.cs)
- 1.1.2.1.2 Subtask: Crear Views/Registro.razor (reemplaza FrmRegistro.cs)
- 1.1.2.1.3 Subtask: Crear Views/Dashboard.razor (reemplaza FrmBienvenida.cs)

- 1.1.2.1.4 Subtask: Crear Views/Alimentos.razor (reemplaza FrmAlimentos.cs y FrmAgregarAlimento.cs)
- 1.1.2.1.5 Subtask: Crear Views/Menus.razor (reemplaza FrmMenus.cs y FrmCrearMenu.cs)
- 1.1.2.1.6 Subtask: Crear Views/Perfil.razor (reemplaza FrmPerfil.cs)
- 1.1.2.1.7 Subtask: Crear Views/Estadisticas.razor (reemplaza FrmEstadisticas.cs)

1.2 Feature: Persistencia de datos con SQLite

1.2.1 PBI: Implementación de base de datos

1.2.1.1 Task: Configuración de SQLite

Descripción:
Implementar SQLite como sistema de persistencia de datos, reemplazando completamente el uso de archivos CSV y estructurando la información en tablas relacionales.

- 1.2.1.1.1 Subtask: Crear Data/DatabaseContext.cs
- 1.2.1.1.2 Subtask: Crear tablas: Usuarios, Perfiles, Alimentos, Menus, ItemsMenu
- 1.2.1.1.3 Subtask: Implementar datos iniciales (seed automático)
- 1.2.1.1.4 Subtask: Eliminar uso de archivos CSV

1.2.2 PBI: Migración de controllers a SQLite

1.2.2.1 Task: Migración de lógica de datos

Descripción:
Actualizar los controllers para que utilicen SQLite en lugar de archivos CSV, manteniendo intactas las interfaces existentes.

- 1.2.2.1.1 Subtask: Migrar LoginController.cs a SQLite
- 1.2.2.1.2 Subtask: Migrar AlimentoController.cs a SQLite
- 1.2.2.1.3 Subtask: Migrar MenuController.cs a SQLite
- 1.2.2.1.4 Subtask: Migrar PerfilController.cs a SQLite
- 1.2.2.1.5 Subtask: Mantener interfaces sin cambios (ILoginController, etc.)

1.3 Feature: Ajustes del modelo y estado de la aplicación

1.3.1 PBI: Actualización del modelo User

1.3.1.1 Task: Extensión del modelo

Descripción:
Actualizar la entidad User para soportar funcionalidades administrativas dentro del sistema.

- 1.3.1.1.1 Subtask: Agregar propiedad IsAdmin
- 1.3.1.1.2 Subtask: Agregar propiedad IsActive

1.3.2 PBI: Manejo de estado de sesión

1.3.2.1 Task: Implementación de estado de sesión

Descripción:
Implementar un mecanismo de manejo de estado en Blazor Server para mantener la información del usuario autenticado durante la sesión.

- 1.3.2.1.1 Subtask: Crear AppState.cs
- 1.3.2.1.2 Subtask: Gestionar usuario logueado por circuito Blazor

EPIC 2. Separación de Controllers

Descripción:
Este Epic tiene como objetivo aplicar principios SOLID en la arquitectura del sistema, específicamente el principio de responsabilidad única (SRP) y el principio de segregación de interfaces (ISP). Se refactoriza el LoginController para separar responsabilidades y mejorar la mantenibilidad, escalabilidad y claridad del código.

2.1 Feature: Refactorización basada en principios SOLID

2.1.1 PBI: Separación de responsabilidades en controllers

2.1.1.1 Task: REFACTOR-1.1 — Separar LoginController en dos clases

Descripción:
El LoginController contenía múltiples responsabilidades relacionadas tanto con

autenticación como con administración de usuarios. Se refactoriza para dividir estas responsabilidades en dos controllers independientes, cumpliendo con los principios SOLID.

- 2.1.1.1.1 Subtask: Crear interfaz IUserManagementController en Controllers/Abstractions/
- 2.1.1.1.2 Subtask: Crear UserManagementController con métodos: GetAll(), ResetPassword(), SetActive(), EliminarUsuario()
- 2.1.1.1.3 Subtask: Refactorizar LoginController, manteniendo solo Login(), Register() y GetUser()
- 2.1.1.1.4 Subtask: Registrar UserManagementController en el contenedor de inyección de dependencias (Program.cs)

EPIC 3. Implementar panel de administración

Descripción:
Este Epic introduce funcionalidades orientadas al rol de administrador dentro del sistema, permitiendo la gestión de usuarios y la visualización de estadísticas globales. Estas funcionalidades están restringidas a usuarios con privilegios administrativos (IsAdmin = true).

3.1 Feature: Interfaz de administración

3.1.1 PBI: Vista de administración

3.1.1.1 Task: ADMIN-1.1 — Vista de administración

Descripción:
Desarrollar la interfaz principal de administración accesible únicamente para usuarios con privilegios de administrador, permitiendo la gestión completa de usuarios.

- 3.1.1.1.1 Subtask: Crear Views/Admin.razor con control de acceso (IsAdmin)
- 3.1.1.1.2 Subtask: Implementar tabla de usuarios con estado activo/inactivo
- 3.1.1.1.3 Subtask: Agregar botón para resetear contraseña (valor por defecto: 1234)
- 3.1.1.1.4 Subtask: Agregar botón para activar/desactivar usuario
- 3.1.1.1.5 Subtask: Agregar botón para eliminar usuario permanentemente

---

3.2 Feature: Estadísticas administrativas

3.2.1 PBI: Visualización de estadísticas globales

3.2.1.1 Task: ADMIN-1.2 — Estadísticas administrativas

Descripción:
Implementar un conjunto de visualizaciones que permitan al administrador analizar el comportamiento general del sistema mediante métricas relevantes.

- 3.2.1.1.1 Subtask: Mostrar tarjetas de resumen (total de usuarios, usuarios activos, alimento más consumido)
- 3.2.1.1.2 Subtask: Mostrar distribución de tipos de dieta con barras de progreso y porcentajes
- 3.2.1.1.3 Subtask: Mostrar Top 5 usuarios con mayor cantidad de menús registrados
- 3.2.1.1.4 Subtask: Implementar buscador de alimento más consumido con filtro por rango de fechas

---

3.3 Feature: Integración en navegación

3.3.1 PBI: Acceso al panel de administración

3.3.1.1 Task: ADMIN-1.3 — Integración en navegación

Descripción:
Actualizar la navegación principal de la aplicación para mostrar el acceso al panel de administración únicamente cuando el usuario autenticado tenga permisos de administrador.

- 3.3.1.1.1 Subtask: Agregar enlace "Admin" en MainLayout.razor condicionado a AppState.IsAdmin
- 3.3.1.1.2 Subtask: Mostrar etiqueta o badge "Admin" junto al nombre del usuario en el navbar

---

EPIC 4. Mejora — Testing y Cobertura de Controllers

Descripción:
Este Epic tiene como objetivo garantizar la calidad y confiabilidad del sistema mediante la implementación de pruebas automatizadas. Se utiliza xUnit junto con una base de datos SQLite en memoria para validar el comportamiento de los controllers de forma aislada, asegurando el correcto funcionamiento de la lógica de negocio.

4.1 Feature: Configuración del entorno de pruebas

4.1.1 PBI: Implementación del proyecto de testing

4.1.1.1 Task: TEST-1 — Configurar proyecto de pruebas xUnit

Descripción:
Crear un proyecto independiente de pruebas utilizando xUnit y SQLite en memoria, permitiendo ejecutar pruebas unitarias aisladas del entorno de producción.

- 4.1.1.1.1 Subtask: Crear NutricionApp.Tests.csproj con dependencias xUnit, coverlet y SQLite
- 4.1.1.1.2 Subtask: Crear TestDatabaseFactory.cs para generar una base de datos en memoria por cada test
- 4.1.1.1.3 Subtask: Actualizar NutricionApp.csproj para excluir NutricionApp.Tests** del build principal
- 4.1.1.1.4 Subtask: Actualizar NutricionApp.sln para incluir ambos proyectos
- 4.1.1.1.5 Subtask: Modificar DatabaseContext.cs agregando constructor con connection string

---

4.2 Feature: Pruebas unitarias de controllers

4.2.1 PBI: Validación funcional del sistema

4.2.1.1 Task: TEST-2 — Tests de LoginController

Descripción:
Validar el correcto funcionamiento de la autenticación y registro de usuarios, asegurando el manejo adecuado de credenciales, estados y roles.

- 4.2.1.1.1 Subtask: Login con credenciales correctas retorna true
- 4.2.1.1.2 Subtask: Login con contraseña incorrecta retorna false
- 4.2.1.1.3 Subtask: Login con usuario inexistente retorna false
- 4.2.1.1.4 Subtask: Login con usuario administrador retorna true
- 4.2.1.1.5 Subtask: Login con usuario inactivo retorna false
- 4.2.1.1.6 Subtask: Register con usuario nuevo retorna true
- 4.2.1.1.7 Subtask: Register con usuario duplicado retorna false
- 4.2.1.1.8 Subtask: Register permite login después del registro
- 4.2.1.1.9 Subtask: GetUser retorna objeto completo con datos correctos
- 4.2.1.1.10 Subtask: GetUser de admin retorna IsAdmin true
- 4.2.1.1.11 Subtask: GetUser de usuario normal retorna IsAdmin false
- 4.2.1.1.12 Subtask: GetUser de usuario inexistente retorna null

---

4.2.1.2 Task: TEST-3 — Tests de AlimentoController

Descripción:
Validar las operaciones CRUD del catálogo de alimentos, asegurando la correcta manipulación y persistencia de los datos.

- 4.2.1.2.1 Subtask: ObtenerTodos retorna lista no vacía
- 4.2.1.2.2 Subtask: ObtenerTodos contiene datos del seed
- 4.2.1.2.3 Subtask: ObtenerTodos está ordenado por nombre
- 4.2.1.2.4 Subtask: ObtenerPorNombre retorna alimento existente
- 4.2.1.2.5 Subtask: ObtenerPorNombre retorna null si no existe
- 4.2.1.2.6 Subtask: ObtenerPorNombre retorna valores nutricionales correctos
- 4.2.1.2.7 Subtask: Agregar nuevo alimento aparece en lista
- 4.2.1.2.8 Subtask: Agregar incrementa conteo de alimentos
- 4.2.1.2.9 Subtask: Actualizar cambia calorías correctamente
- 4.2.1.2.10 Subtask: EliminarPorNombre elimina el alimento
- 4.2.1.2.11 Subtask: EliminarPorNombre decrementa el conteo

4.2.1.3 Task: TEST-4 — Tests de PerfilController

Descripción:
Validar la gestión de perfiles nutricionales, incluyendo la creación, actualización y consulta de información relacionada con dietas y objetivos.

- 4.2.1.3.1 Subtask: ObtenerPerfil con usuario que tiene perfil retorna datos correctos
- 4.2.1.3.2 Subtask: ObtenerPerfil con usuario sin perfil retorna perfil default
- 4.2.1.3.3 Subtask: ObtenerPerfil retorna dieta correcta
- 4.2.1.3.4 Subtask: ObtenerPerfil retorna objetivo correcto
- 4.2.1.3.5 Subtask: GuardarPerfil nuevo se guarda correctamente
- 4.2.1.3.6 Subtask: GuardarPerfil actualiza perfil existente
- 4.2.1.3.7 Subtask: GuardarPerfil cambio de dieta se refleja
- 4.2.1.3.8 Subtask: DistribucionDietas retorna lista no vacía
- 4.2.1.3.9 Subtask: DistribucionDietas contiene los 3 tipos del seed

---

4.2.1.4 Task: TEST-5 — Tests de MenuController

Descripción:
Validar el registro, almacenamiento y consulta de menús diarios, asegurando la integridad de los datos y la correcta generación de estadísticas.

- 4.2.1.4.1 Subtask: ObtenerPorUsuario sin menús retorna lista vacía
- 4.2.1.4.2 Subtask: ObtenerPorUsuario con menú guardado lo retorna
- 4.2.1.4.3 Subtask: Guardar menú con items se guarda correctamente
- 4.2.1.4.4 Subtask: Guardar varios menús los guarda todos
- 4.2.1.4.5 Subtask: Guardar items con calorías correctas
- 4.2.1.4.6 Subtask: Eliminar menú existente lo elimina
- 4.2.1.4.7 Subtask: AlimentoMasConsumido sin datos retorna "Sin datos"
- 4.2.1.4.8 Subtask: AlimentoMasConsumido con datos retorna el más consumido
- 4.2.1.4.9 Subtask: TopUsuariosPorMenus sin menús retorna vacío
- 4.2.1.4.10 Subtask: TopUsuariosPorMenus ordenado descendente

---

4.2.1.5 Task: TEST-6 — Tests de UserManagementController

Descripción:
Validar las funcionalidades administrativas relacionadas con la gestión de usuarios, asegurando el correcto control de acceso y estado de las cuentas.

- 4.2.1.5.1 Subtask: GetAll retorna lista no vacía
- 4.2.1.5.2 Subtask: GetAll contiene admin
- 4.2.1.5.3 Subtask: GetAll contiene todos los usuarios del seed

- 4.2.1.5.4 Subtask: ResetPassword nueva contraseña funciona
- 4.2.1.5.5 Subtask: ResetPassword contraseña anterior ya no funciona
- 4.2.1.5.6 Subtask: SetActive desactivar usuario no puede hacer login
- 4.2.1.5.7 Subtask: SetActive reactivar usuario puede hacer login nuevamente
- 4.2.1.5.8 Subtask: SetActive desactivar aparece flag en GetAll
- 4.2.1.5.9 Subtask: EliminarUsuario no aparece en GetAll
- 4.2.1.5.10 Subtask: EliminarUsuario no puede hacer login
- 4.2.1.5.11 Subtask: EliminarUsuario decrementa conteo de usuarios
-

EPIC 5. Descarga de Reportes

Descripción:
Este Epic incorpora funcionalidades que permiten a los usuarios exportar información del sistema en formato CSV, facilitando el análisis externo de los datos.

5.1 Feature: Exportación desde Menús

5.1.1 PBI: Descarga CSV

5.1.1.1 Task: CSV-1

Descripción:
Permitir la descarga de información de menús en formato CSV desde la interfaz del usuario.

- 5.1.1.1.1 Subtask: Agregar botón CSV
- 5.1.1.1.2 Subtask: Implementar descarga
- 5.1.1.1.3 Subtask: Generar archivo
- 5.1.1.1.4 Subtask: Compatibilidad con navegadores

5.2 Feature: Exportación desde Estadísticas

5.2.1 PBI: Descarga CSV en estadísticas

5.2.1.1 Task: CSV-2

Descripción:
Permitir la exportación de datos estadísticos en formato CSV desde diferentes secciones del sistema.

- 5.2.1.1.1 Subtask: Botón en resumen diario
- 5.2.1.1.2 Subtask: Botón en búsqueda por fecha
- 5.2.1.1.3 Subtask: Botón en resumen mensual
- 5.2.1.1.4 Subtask: Método reutilizable
- 5.2.1.1.5 Subtask: Método de descarga
- 5.2.1.1.6 Subtask: Nombres de archivo descriptivos

Conclusiones / Análisis de Aprendizaje

La implementación de las mejoras permitió mejorar significativamente la arquitectura, mantenibilidad y calidad del sistema. La separación de responsabilidades mediante el patrón Repository, junto con la incorporación de pruebas automatizadas, incrementó la confiabilidad del sistema.

Asimismo, la implementación de funcionalidades como la descarga de reportes en CSV aporta valor al usuario final. En conjunto, el proyecto refleja la aplicación de buenas prácticas de ingeniería de software en un entorno real.